CardDemo — Software Requirements Specification

Derived from the COBOL/CICS/VSAM source for the Rust re-implementation

Illia Burov (prepared with Claude Code)

2026-11-06 — version 1.2

Contents

1 Introduction **2**
1.1 Purpose . . . 2
1.2 Scope . . . 2
1.3 Conventions . . . 2
1.4 System overview . . . 2
1.5 Data stores . . . 3

2 Data Dictionary **3**

3 Online Requirements — Common Behaviour, Sign-on, Menus, User Administration **9**
3.1 Common online behaviour (BR-ONL) . . . 9
3.2 FR-SGN — Sign-on (CC00, COSGN00C, map COSGN00/COSGN0A) . . . 10
3.3 FR-MEN — Main menu (CM00, COMEN01C, map COMEN01/COMEN1A) . . . 10
3.4 FR-ADM — Admin menu (CA00, COADM01C, map COADM01/COADM1A) . . . 11
3.5 FR-USR — User administration (security file USRSEC, record CSUSR01Y: id 8, first name 20, last name 20, password 8, type 1, filler 23) . . . 11

4 Online Requirements — Accounts and Cards **13**
4.1 FR-ACV — Account view (CAVW, COACTVWC, map COACTVW/CACTVWA) . . . 13
4.2 FR-CDV — Credit card detail view (CCDL, COCRDSLC, map COCRDSL/CCRDSLA) . . . 14
4.3 FR-CDL — Credit card list (CCLI, COCRDLIC, map COCRDLI/CCRDLIA) . . . 14
4.4 FR-CDU — Credit card update (CCUP, COCRDUPC, map COCRDUP/CCRDUPA) . . . 15
4.5 FR-ACU — Account update (CAUP, COACTUPC, map COACTUP/CACTUPA) . . . 16

5 Online Requirements — Transactions, Bill Payment, Reports **18**
5.1 FR-TRL — Transaction list (CT00, COTRN00C, map COTRN00/COTRN0A) . . . 18
5.2 FR-TRV — Transaction view (CT01, COTRN01C, map COTRN01/COTRN1A) . . . 19
5.3 FR-TRA — Transaction add (CT02, COTRN02C, map COTRN02/COTRN2A) . . . 19
5.4 FR-BIL — Bill payment (CB00, COBIL00C, map COBIL00/COBIL0A) . . . 20
5.5 FR-RPT — Transaction reports (CR00, CORPT00C, map CORPT00/CORPT0A) . . . 20

6 Batch Processing Requirements **21**
6.1 FR-BAT-POST — Daily transaction posting (job POSTTRAN, program CBTRN02C) . . . 21
6.2 FR-BAT-INT — Interest calculation (job INTCALC, program CBACT04C) . . . 22
6.3 FR-BAT-REPT — Transaction detail report (job TRANREPT, program CBTRN03C) . . . 23
6.4 FR-BAT-STMT — Customer statements (job CREASTMT, programs CBSTM03A + CBSTM03B) . . . 23
6.5 FR-BAT-RDR — File dump utilities (READACCT/CBACT01C, READCARD/CBACT02C, READXREF/CBACT03C, READCUST/CBCUS01C) . . . 24
6.6 FR-BAT-VER — Daily transaction verification (CBTRN01C; no JCL ships with the original) . . . 24
6.7 FR-BAT-XPRT — Export / import (CBEXPORT, CBIMPORT) . . . 25
6.8 FR-BAT-UTIL — Utilities . . . 25

6.9 Known defects kept for fidelity 26

7 Non-Functional Requirements **26**

8 Traceability and Glossary **27**
8.1 Program → requirement traceability 27
8.2 Glossary 28

1 Introduction

1.1 Purpose

This document gives the functional and non-functional requirements of the **CardDemo** credit-card management application. It was produced by reverse-engineering the original mainframe source code (COBOL, CICS BMS maps, copybooks, JCL and assembler) in the repository `aws-mainframe-modernization-carddemo`.

It serves three purposes:

1. It is the specification for the Rust re-implementation (`rust/`).
2. It is the basis of the manual test suite. Every test case traces to a requirement ID defined here.
3. It is a record of how the system behaves today, **including its defects**. The re-implementation keeps those defects on purpose (they are listed as *KD-nn*), so that the old and new systems can be compared output for output.

1.2 Scope

The scope is the **base application** only:

- 17 online CICS programs
- 13 batch COBOL programs plus 2 assembler routines
- the VSAM data model
- the batch job streams

The optional modules are **out of scope**: credit-card authorisations (IMS/DB2/MQ), transaction type management (DB2), and the MQ account/date inquiry. Where a base program refers to an optional module, these requirements describe what the base program does when that module is *not installed*:

- main menu option 11
- admin menu options 5–6

1.3 Conventions

Prefix	Meaning
`BR-…`	Business or behavioural rule shared by several functions
`FR-⟨AREA⟩-nn`	Functional requirement
`NFR-nn`	Non-functional requirement
`KD-nn`	Known defect of the original, kept for fidelity

Text in `monospace` is literal: messages exactly as the user sees them, field values, file and program names. Every requirement names its source program, so it can be traced back to the code.

1.4 System overview

Area	Transactions / jobs	Programs
Security	CC00 sign-on; CU00–CU03 user list/add/update/delete	COSGN00C, COUSR00C–03C
Navigation	CM00 main menu; CA00 admin menu	COMEN01C, COADM01C
Accounts	CAVW view; CAUP update	COACTVWC, COACTUPC
Cards	CCLI list; CCDL view; CCUP update	COCRDLIC, COCRDSLC, COCRDUPC
Transactions	CT00 list; CT01 view; CT02 add; CR00 reports; CB00 bill pay	COTRN00C–02C, CORPT00C, COBIL00C
Batch	POSTTRAN, INTCALC, TRANREPT, CREASTMT, COMBTRAN, READxxxx, CBEXPORT/CBIMPORT, WAITSTEP, file (re)load jobs	CBTRN01C–03C, CBACT01C–04C, CBCUS01C, CBSTM03A/B, CBEXPORT, CBIMPORT, COBSWAIT, CSUTLDTC, COBDATFT, MVSWAIT

The two user roles are **Admin** (`A`), who lands on the admin menu after sign-on, and **User** (`U`), who lands on the main menu.

1.5 Data stores

Store (CICS file)	Organisation	Key	Record
USRSEC	KSDS	user id (8)	SEC-USER-DATA (CSUSR01Y)
ACCTDAT	KSDS	account id 9(11)	ACCOUNT-RECORD (CVACT01Y)
CARDDAT	KSDS + AIX CARDAIX on account id	card number (16)	CARD-RECORD (CVACT02Y)
CUSTDAT	KSDS	customer id 9(9)	CUSTOMER-RECORD (CVCUS01Y)
CCXREF	KSDS + AIX CXACAIX on account id	card number (16)	CARD-XREF-RECORD (CVACT03Y)
TRANSACT	KSDS (+ AIX on proc timestamp)	tran id (16)	TRAN-RECORD (CVTRA05Y)
TCATBALF	KSDS	acct+type+cat (17)	TRAN-CAT-BAL-RECORD (CVTRA01Y)
DISCGRP	KSDS	group+type+cat (16)	DIS-GROUP-RECORD (CVTRA02Y)
TRANTYPE	KSDS	type (2)	TRAN-TYPE-RECORD (CVTRA03Y)
TRANCATG	KSDS	type+cat (6)	TRAN-CAT-RECORD (CVTRA04Y)
DALYTRAN	sequential	—	DALYTRAN-RECORD (CVTRA06Y)

The full field-level layout of every record is in the *Data Dictionary* chapter.

2 Data Dictionary

Field layouts generated from the copybooks by `rust/tools/copybook.py`. Offsets are 1-based. COMP/COMP-3 lengths are storage bytes.

2.0.1 SEC-USER-DATA — `CSUSR01Y.cpy` (80 bytes)

Offset	Len	Field	Picture	Notes
1	8	SEC-USR-ID	X(08)	
9	20	SEC-USR-FNAME	X(20)	

Offset	Len	Field	Picture	Notes
29	20	SEC-USR-LNAME	X(20)	
49	8	SEC-USR-PWD	X(08)	
57	1	SEC-USR-TYPE	X(01)	
58	23	SEC-USR-FILLER	X(23)	

2.0.2 ACCOUNT-RECORD — `CVACT01Y.cpy` (300 bytes)

Offset	Len	Field	Picture	Notes
1	11	ACCT-ID	9(11)	
12	1	ACCT-ACTIVE-STATUS	X(01)	
13	12	ACCT-CURR-BAL	S9(10)V99	
25	12	ACCT-CREDIT-LIMIT	S9(10)V99	
37	12	ACCT-CASH-CREDIT-LIMIT	S9(10)V99	
49	10	ACCT-OPEN-DATE	X(10)	
59	10	ACCT-EXPIRAION-DATE	X(10)	
69	10	ACCT-REISSUE-DATE	X(10)	
79	12	ACCT-CURR-CYC-CREDIT	S9(10)V99	
91	12	ACCT-CURR-CYC-DEBIT	S9(10)V99	
103	10	ACCT-ADDR-ZIP	X(10)	
113	10	ACCT-GROUP-ID	X(10)	
123	178	FILLER	X(178)	

2.0.3 CARD-RECORD — `CVACT02Y.cpy` (150 bytes)

Offset	Len	Field	Picture	Notes
1	16	CARD-NUM	X(16)	
17	11	CARD-ACCT-ID	9(11)	
28	3	CARD-CVV-CD	9(03)	
31	50	CARD-EMBOSSED-NAME	X(50)	
81	10	CARD-EXPIRAION-DATE	X(10)	
91	1	CARD-ACTIVE-STATUS	X(01)	
92	59	FILLER	X(59)	

2.0.4 CARD-XREF-RECORD — `CVACT03Y.cpy` (50 bytes)

Offset	Len	Field	Picture	Notes
1	16	XREF-CARD-NUM	X(16)	
17	9	XREF-CUST-ID	9(09)	
26	11	XREF-ACCT-ID	9(11)	
37	14	FILLER	X(14)	

2.0.5 CUSTOMER-RECORD — `CVCUS01Y.cpy` (500 bytes)

Offset	Len	Field	Picture	Notes
1	9	CUST-ID	9(09)	
10	25	CUST-FIRST-NAME	X(25)	
35	25	CUST-MIDDLE-NAME	X(25)	
60	25	CUST-LAST-NAME	X(25)	
85	50	CUST-ADDR-LINE-1	X(50)	
135	50	CUST-ADDR-LINE-2	X(50)	
185	50	CUST-ADDR-LINE-3	X(50)	
235	2	CUST-ADDR-STATE-CD	X(02)	
237	3	CUST-ADDR-COUNTRY-CD	X(03)	
240	10	CUST-ADDR-ZIP	X(10)	
250	15	CUST-PHONE-NUM-1	X(15)	
265	15	CUST-PHONE-NUM-2	X(15)	
280	9	CUST-SSN	9(09)	
289	20	CUST-GOVT-ISSUED-ID	X(20)	
309	10	CUST-DOB-YYYY-MM-DD	X(10)	
319	10	CUST-EFT-ACCOUNT-ID	X(10)	
329	1	CUST-PRI-CARD-HOLDER-IND	X(01)	
330	3	CUST-FICO-CREDIT-SCORE	9(03)	
333	168	FILLER	X(168)	

2.0.6 **TRAN-CAT-BAL-RECORD** — `CVTRA01Y.cpy` **(50 bytes)**

Offset	Len	Field	Picture	Notes
1	17	TRAN-CAT-KEY	group	
1	11	TRANCAT-ACCT-ID	9(11)	
12	2	TRANCAT-TYPE-CD	X(02)	
14	4	TRANCAT-CD	9(04)	
18	11	TRAN-CAT-BAL	S9(09)V99	
29	22	FILLER	X(22)	

2.0.7 **DIS-GROUP-RECORD** — `CVTRA02Y.cpy` **(50 bytes)**

Offset	Len	Field	Picture	Notes
1	16	DIS-GROUP-KEY	group	
1	10	DIS-ACCT-GROUP-ID	X(10)	
11	2	DIS-TRAN-TYPE-CD	X(02)	
13	4	DIS-TRAN-CAT-CD	9(04)	
17	6	DIS-INT-RATE	S9(04)V99	
23	28	FILLER	X(28)	

2.0.8 **TRAN-TYPE-RECORD** — `CVTRA03Y.cpy` **(60 bytes)**

Offset	Len	Field	Picture	Notes
1	2	TRAN-TYPE	X(02)	
3	50	TRAN-TYPE-DESC	X(50)	
53	8	FILLER	X(08)	

2.0.9 TRAN-CAT-RECORD — `CVTRA04Y.cpy` (60 bytes)

Offset	Len	Field	Picture	Notes
1	6	TRAN-CAT-KEY	group	
1	2	TRAN-TYPE-CD	X(02)	
3	4	TRAN-CAT-CD	9(04)	
7	50	TRAN-CAT-TYPE-DESC	X(50)	
57	4	FILLER	X(04)	

2.0.10 TRAN-RECORD — `CVTRA05Y.cpy` (350 bytes)

Offset	Len	Field	Picture	Notes
1	16	TRAN-ID	X(16)	
17	2	TRAN-TYPE-CD	X(02)	
19	4	TRAN-CAT-CD	9(04)	
23	10	TRAN-SOURCE	X(10)	
33	100	TRAN-DESC	X(100)	
133	11	TRAN-AMT	S9(09)V99	
144	9	TRAN-MERCHANT-ID	9(09)	
153	50	TRAN-MERCHANT-NAME	X(50)	
203	50	TRAN-MERCHANT-CITY	X(50)	
253	10	TRAN-MERCHANT-ZIP	X(10)	
263	16	TRAN-CARD-NUM	X(16)	
279	26	TRAN-ORIG-TS	X(26)	
305	26	TRAN-PROC-TS	X(26)	
331	20	FILLER	X(20)	

2.0.11 DALYTRAN-RECORD — `CVTRA06Y.cpy` (350 bytes)

Offset	Len	Field	Picture	Notes
1	16	DALYTRAN-ID	X(16)	
17	2	DALYTRAN-TYPE-CD	X(02)	
19	4	DALYTRAN-CAT-CD	9(04)	
23	10	DALYTRAN-SOURCE	X(10)	
33	100	DALYTRAN-DESC	X(100)	
133	11	DALYTRAN-AMT	S9(09)V99	
144	9	DALYTRAN-MERCHANT-ID	9(09)	
153	50	DALYTRAN-MERCHANT-NAME	X(50)	
203	50	DALYTRAN-MERCHANT-CITY	X(50)	

Offset	Len	Field	Picture	Notes
253	10	DALYTRAN-MERCHANT-ZIP	X(10)	
263	16	DALYTRAN-CARD-NUM	X(16)	
279	26	DALYTRAN-ORIG-TS	X(26)	
305	26	DALYTRAN-PROC-TS	X(26)	
331	20	FILLER	X(20)	

2.0.12 **TRNX-RECORD** — `COSTM01.CPY` **(350 bytes)**

Offset	Len	Field	Picture	Notes
1	32	TRNX-KEY	group	
1	16	TRNX-CARD-NUM	X(16)	
17	16	TRNX-ID	X(16)	
33	318	TRNX-REST	group	
33	2	TRNX-TYPE-CD	X(02)	
35	4	TRNX-CAT-CD	9(04)	
39	10	TRNX-SOURCE	X(10)	
49	100	TRNX-DESC	X(100)	
149	11	TRNX-AMT	S9(09)V99	
160	9	TRNX-MERCHANT-ID	9(09)	
169	50	TRNX-MERCHANT-NAME	X(50)	
219	50	TRNX-MERCHANT-CITY	X(50)	
269	10	TRNX-MERCHANT-ZIP	X(10)	
279	26	TRNX-ORIG-TS	X(26)	
305	26	TRNX-PROC-TS	X(26)	
331	20	FILLER	X(20)	

2.0.13 **EXPORT-RECORD** — `CVEXPORT.cpy` **(500 bytes)**

Offset	Len	Field	Picture	Notes
1	1	EXPORT-REC-TYPE	X(1)	
2	26	EXPORT-TIMESTAMP	X(26)	
2	26	EXPORT-TIMESTAMP-R	group	REDEFINES EXPORT-TIMESTAMP
2	10	EXPORT-DATE	X(10)	
12	1	EXPORT-DATE-TIME-SEP	X(1)	
13	15	EXPORT-TIME	X(15)	
28	4	EXPORT-SEQUENCE-NUM	9(9) COMP	
32	4	EXPORT-BRANCH-ID	X(4)	
36	5	EXPORT-REGION-CODE	X(5)	
41	460	EXPORT-RECORD-DATA	X(460)	
41	460	EXPORT-CUSTOMER-DATA	group	REDEFINES EXPORT-RECORD-DATA
41	4	EXP-CUST-ID	9(09) COMP	
45	25	EXP-CUST-FIRST-NAME	X(25)	
70	25	EXP-CUST-MIDDLE-NAME	X(25)	
95	25	EXP-CUST-LAST-NAME	X(25)	
120	50	EXP-CUST-ADDR-LINES	group	OCCURS 3

Offset	Len	Field	Picture	Notes
120	50	EXP-CUST-ADDR-LINE	X(50)	
270	2	EXP-CUST-ADDR-STATE-CD	X(02)	
272	3	EXP-CUST-ADDR-COUNTRY-CD	X(03)	
275	10	EXP-CUST-ADDR-ZIP	X(10)	
285	15	EXP-CUST-PHONE-NUMS	group	OCCURS 2
285	15	EXP-CUST-PHONE-NUM	X(15)	
315	9	EXP-CUST-SSN	9(09)	
324	20	EXP-CUST-GOVT-ISSUED-ID	X(20)	
344	10	EXP-CUST-DOB-YYYY-MM-DD	X(10)	
354	10	EXP-CUST-EFT-ACCOUNT-ID	X(10)	
364	1	EXP-CUST-PRI-CARD-HOLDER-IND	X(01)	
365	2	EXP-CUST-FICO-CREDIT-SCORE	9(03) COMP-3	
367	134	FILLER	X(134)	
41	460	EXPORT-ACCOUNT-DATA	group	REDEFINES EXPORT-RECORD-DATA
41	11	EXP-ACCT-ID	9(11)	
52	1	EXP-ACCT-ACTIVE-STATUS	X(01)	
53	7	EXP-ACCT-CURR-BAL	S9(10)V99 COMP-3	
60	12	EXP-ACCT-CREDIT-LIMIT	S9(10)V99	
72	7	EXP-ACCT-CASH-CREDIT-LIMIT	S9(10)V99 COMP-3	
79	10	EXP-ACCT-OPEN-DATE	X(10)	
89	10	EXP-ACCT-EXPIRAION-DATE	X(10)	
99	10	EXP-ACCT-REISSUE-DATE	X(10)	
109	12	EXP-ACCT-CURR-CYC-CREDIT	S9(10)V99	
121	8	EXP-ACCT-CURR-CYC-DEBIT	S9(10)V99 COMP	
129	10	EXP-ACCT-ADDR-ZIP	X(10)	
139	10	EXP-ACCT-GROUP-ID	X(10)	
149	352	FILLER	X(352)	
41	460	EXPORT-TRANSACTION-DATA	group	REDEFINES EXPORT-RECORD-DATA
41	16	EXP-TRAN-ID	X(16)	
57	2	EXP-TRAN-TYPE-CD	X(02)	
59	4	EXP-TRAN-CAT-CD	9(04)	
63	10	EXP-TRAN-SOURCE	X(10)	
73	100	EXP-TRAN-DESC	X(100)	
173	6	EXP-TRAN-AMT	S9(09)V99 COMP-3	
179	4	EXP-TRAN-MERCHANT-ID	9(09) COMP	
183	50	EXP-TRAN-MERCHANT-NAME	X(50)	
233	50	EXP-TRAN-MERCHANT-CITY	X(50)	
283	10	EXP-TRAN-MERCHANT-ZIP	X(10)	
293	16	EXP-TRAN-CARD-NUM	X(16)	
309	26	EXP-TRAN-ORIG-TS	X(26)	
335	26	EXP-TRAN-PROC-TS	X(26)	
361	140	FILLER	X(140)	
41	460	EXPORT-CARD-XREF-DATA	group	REDEFINES EXPORT-RECORD-DATA
41	16	EXP-XREF-CARD-NUM	X(16)	
57	9	EXP-XREF-CUST-ID	9(09)	
66	8	EXP-XREF-ACCT-ID	9(11) COMP	
74	427	FILLER	X(427)	

Offset	Len	Field	Picture	Notes
41	460	EXPORT-CARD-DATA	group	REDEFINES EXPORT-RECORD-DATA
41	16	EXP-CARD-NUM	X(16)	
57	8	EXP-CARD-ACCT-ID	9(11) COMP	
65	2	EXP-CARD-CVV-CD	9(03) COMP	
67	50	EXP-CARD-EMBOSSED-NAME	X(50)	
117	10	EXP-CARD-EXPIRAION-DATE	X(10)	
127	1	EXP-CARD-ACTIVE-STATUS	X(01)	
128	373	FILLER	X(373)	

2.0.14 CARDDEMO-COMMAREA — `COCOM01Y.cpy` (160 bytes)

Offset	Len	Field	Picture	Notes
1	34	CDEMO-GENERAL-INFO	group	
1	4	CDEMO-FROM-TRANID	X(04)	
5	8	CDEMO-FROM-PROGRAM	X(08)	
13	4	CDEMO-TO-TRANID	X(04)	
17	8	CDEMO-TO-PROGRAM	X(08)	
25	8	CDEMO-USER-ID	X(08)	
33	1	CDEMO-USER-TYPE	X(01)	
34	1	CDEMO-PGM-CONTEXT	9(01)	
35	84	CDEMO-CUSTOMER-INFO	group	
35	9	CDEMO-CUST-ID	9(09)	
44	25	CDEMO-CUST-FNAME	X(25)	
69	25	CDEMO-CUST-MNAME	X(25)	
94	25	CDEMO-CUST-LNAME	X(25)	
119	12	CDEMO-ACCOUNT-INFO	group	
119	11	CDEMO-ACCT-ID	9(11)	
130	1	CDEMO-ACCT-STATUS	X(01)	
131	16	CDEMO-CARD-INFO	group	
131	16	CDEMO-CARD-NUM	9(16)	
147	14	CDEMO-MORE-INFO	group	
147	7	CDEMO-LAST-MAP	X(7)	
154	7	CDEMO-LAST-MAPSET	X(7)	

3 Online Requirements — Common Behaviour, Sign-on, Menus, User Administration

3.1 Common online behaviour (BR-ONL)

ID	Requirement
BR-ONL-001	Every screen is a 24×80 map. Header: row 1 `Tran :` + transaction id (col 8), title line 1 (col 21, yellow), `Date :` + `mm/dd/yy` (col 71). Row 2 `Prog :` + program name, title line 2, `Time :` + `hh:mm:ss`. Titles: `AWS Mainframe Modernization` and `CardDemo` (COTTL01Y).

ID	Requirement
BR-ONL-002	Interaction is pseudo-conversational. A program sends its screen and ends with `RETURN TRANSID(<own tranid>) COMMAREA(...)`, and the next key press re-enters the same program. The first entry is detected with `CDEMO-PGM-CONTEXT = 0` (ENTER), and the program sets it to 1 (REENTER) before sending.
BR-ONL-003	Navigating to another program uses XCTL. The caller sets `CDEMO-FROM-TRANID`, `CDEMO-FROM-PROGRAM`, `CDEMO-TO-PROGRAM` and `CDEMO-PGM-CONTEXT = 0`.
BR-ONL-004	If a program is started without a commarea (EIBCALEN = 0), it transfers to sign-on (COSGN00C). The exception is COSGN00C itself, which shows the sign-on screen.
BR-ONL-005	Any key the screen does not handle shows `Invalid key pressed. Please see below...` (CSMSG01Y) in the message line, and the screen is redisplayed.
BR-ONL-006	Field-level errors: the message goes to the error line (row 23, red unless stated otherwise) and the cursor is placed on the field in error.
BR-ONL-007	The commarea (COCOM01Y, 160 bytes) holds from/to tran and program, user id, user type (`A` admin / `U` user), program context, customer id and names, account id and status, card number, and last map/mapset. Programs append their own working fields after these 160 bytes.
BR-ONL-008	Exiting the application (PF3 on sign-on) clears the screen and shows `Thank you for using CardDemo application....` The task ends without a next transaction.

3.2 FR-SGN — Sign-on (CC00, COSGN00C, map COSGN00/COSGN0A)

ID	Requirement
FR-SGN-01	First entry: show an empty sign-on screen with the cursor in User ID. The screen also shows the application id and system id.
FR-SGN-02	ENTER with a blank user id → `Please enter User ID ...` (cursor User ID). With a blank password → `Please enter Password ...` (cursor Password). *Note:* the password field is defined with FSET and `INITIAL='________'`. If the user does not erase it, the underscores are transmitted and the result is `Wrong Password. Try again ....`
FR-SGN-03	The user id and password are converted to upper case before lookup and comparison.
FR-SGN-04	Read USRSEC by user id. If it is not found → `User not found. Try again ....` Any other error → `Unable to verify the User ....`
FR-SGN-05	Wrong password → `Wrong Password. Try again ...` (cursor Password).
FR-SGN-06	Success: set commarea from-tran `CC00`, from-program `COSGN00C`, user id, user type, context 0. Then XCTL to **COADM01C** if the type is `A`, otherwise to **COMEN01C**.
FR-SGN-07	PF3 → thank-you message, end of session (BR-ONL-008). Any other key → invalid-key message.

3.3 FR-MEN — Main menu (CM00, COMEN01C, map COMEN01/COMEN1A)

ID	Requirement
FR-MEN-01	Show options 1–11 from COMEN02Y as `NN. <name>`: 1 Account View (COACTVWC), 2 Account Update (COACTUPC), 3 Credit Card List (COCRDLIC), 4 Credit Card View (COCRDSLC), 5 Credit Card Update (COCRDUPC), 6 Transaction List (COTRN00C), 7 Transaction View (COTRN01C), 8 Transaction Add (COTRN02C), 9 Transaction Reports (CORPT00C), 10 Bill Payment (COBIL00C), 11 Pending Authorization View (COPAUS0C).
FR-MEN-02	The option input (2 characters) is right-justified and zero-filled. Non-numeric, 0, or greater than 11 → `Please enter a valid option number....`

ID	Requirement
FR-MEN-03	An option flagged admin-only (`A`) chosen by a regular user → `No access - Admin Only option ...`. (No base option is flagged `A`.)
FR-MEN-04	Option 11 (COPAUS0C) first checks that the program is installed. *In the base scope it is not installed*, so the message is `This option Pending Authorization View is not installed ...` (red): the option name up to its first two consecutive spaces.
FR-MEN-05	A program name starting with `DUMMY` → `This option <name-to-first-space>is coming soon ...` (green).
FR-MEN-06	Any other valid option: XCTL to that program with from-tran `CM00`, from-program `COMEN01C`, context 0.
FR-MEN-07	PF3 → XCTL to sign-on (COSGN00C).

3.4 FR-ADM — Admin menu (CA00, COADM01C, map COADM01/COADM1A)

ID	Requirement
FR-ADM-01	Show options 1–6 from COADM02Y: 1 User List (Security), 2 User Add (Security), 3 User Update (Security), 4 User Delete (Security), 5 Transaction Type List/Update (Db2), 6 Transaction Type Maintenance (Db2).
FR-ADM-02	Option validation is the same as FR-MEN-02, with a maximum of 6.
FR-ADM-03	A valid option XCTLs to its program (from-tran `CA00`, from-program `COADM01C`). If the program does not exist (PGMIDERR, *which is the case for options 5–6 in the base scope*), show `This option is not installed ...` in green.
FR-ADM-04	PF3 → sign-on.

3.5 FR-USR — User administration (security file USRSEC, record CSUSR01Y: id 8, first name 20, last name 20, password 8, type 1, filler 23)

3.5.1 FR-USR-LST — User list (CU00, COUSR00C, map COUSR00/COUSR0A)

ID	Requirement
FR-USR-LST-01	Show up to 10 users per page in user-id order: select field, user id, first name, last name, type. The page number is shown. Optional "Search User ID" field: browsing starts at that key, or at the first record if it is blank.
FR-USR-LST-02	First entry and ENTER: read the first page starting at the search key (or the start of the file) and set the page number to 1.
FR-USR-LST-03	Selection: the first row with a non-blank select code is used. `U`/`u` → XCTL COUSR02C (update); `D`/`d` → XCTL COUSR03C (delete). The selected user id is passed in the commarea extension (CDEMO-CU00-USR-SELECTED). Any other code → `Invalid selection. Valid values are U and D`.
FR-USR-LST-04	PF8 (page down): if there is a next page, continue after the last user id shown; otherwise `You are already at the bottom of the page ...`.
FR-USR-LST-05	PF7 (page up): if the page number is greater than 1, page back from the first user id shown; otherwise `You are already at the top of the page ...`.
FR-USR-LST-06	Hitting the end of the file while filling a page → `You have reached the bottom of the page ...` (or `...top of the page ...` going backward). The next-page flag is computed by reading one record ahead.
KD-10	ENDBR is issued without RESP. If STARTBR found nothing (search key beyond the last user), CICS raises INVREQ and the task **abends AEIP**. The same applies to the transaction list (COTRN00C).

ID	Requirement
KD-11	PF7 on page 1 sets NEXT-PAGE-YES. A following PF8 therefore blanks the rows before reaching end of file and shows `You have reached the bottom of the page...` with an empty list. ENTER re-lists.
KD-14	Admin menu (COADM01C): before the XCTL for an option it sets CDEMO-PGM-CONTEXT to 0. For options 5 and 6 (Db2 programs, not installed) the XCTL fails, HANDLE CONDITION PGMIDERR sends `This option is not installed ...` and returns that commarea, so the next Enter is treated as a first entry: the menu is redrawn and the input is ignored once.
FR-USR-LST-07	PF3 → admin menu (COADM01C). File errors → `Unable to lookup User....`

3.5.2 FR-USR-ADD — Add user (CU01, COUSR01C, map COUSR01/COUSR1A)

ID	Requirement
FR-USR-ADD-01	Fields: first name, last name, user id, password, user type. Validation runs in that order, and the first empty field gives `First Name can NOT be empty...`, `Last Name can NOT be empty...`, `User ID can NOT be empty...`, `Password can NOT be empty...` or `User Type can NOT be empty....`
FR-USR-ADD-02	Write the USRSEC record (no case conversion). Success → all fields cleared, and the message `User <id> has been added ...` in green.
FR-USR-ADD-03	Duplicate key → `User ID already exist....` Other error → `Unable to Add User....`
FR-USR-ADD-04	PF4 clears the screen. PF3 → admin menu.

3.5.3 FR-USR-UPD — Update user (CU02, COUSR02C, map COUSR02/COUSR2A)

ID	Requirement
FR-USR-UPD-01	On first entry with a user preselected from the list, fetch that user automatically. ENTER fetches the user in the User ID field: blank → `User ID can NOT be empty...`; not found → `User ID NOT found...`; found → show the fields with `Press PF5 key to save your updates ...` (neutral colour).
FR-USR-UPD-02	PF5 validates all fields (user id, first name, last name, password, type; same messages as FR-USR-ADD-01). It re-reads the record and compares each field. If nothing changed → `Please modify to update ...` (red). Otherwise REWRITE → `User <id> has been updated ...` (green).
FR-USR-UPD-03	PF3 = save (as PF5), then return to the calling program (or the admin menu if none). PF12 = return to the admin menu without saving. PF4 = clear.
FR-USR-UPD-04	Errors: `Unable to lookup User...`, `Unable to Update User....`

3.5.4 FR-USR-DEL — Delete user (CU03, COUSR03C, map COUSR03/COUSR3A)

ID	Requirement
FR-USR-DEL-01	On first entry with a user preselected from the list, fetch it. ENTER fetches by the User ID field: blank → `User ID can NOT be empty...`; not found → `User ID NOT found...`; found → show first name, last name and type with `Press PF5 key to delete this user ...` (neutral). The password is never shown.
FR-USR-DEL-02	PF5: validate the user id, read the record for update, then DELETE. Success → fields cleared, and `User <id> has been deleted ...` (green). Not found → `User ID NOT found....` Other error → `Unable to Update User...` (literal text of the original).
FR-USR-DEL-03	PF3 → return to the calling program (or the admin menu). PF12 → admin menu. PF4 → clear.

4 Online Requirements — Accounts and Cards

These five programs (COACTVWC, COACTUPC, COCRDLIC, COCRDSLC, COCRDUPC) use a newer coding style than the rest of the application.

They share these rules:

- **BR-AC-01.** Function keys are normalised: PF13–PF24 count as PF1–PF12 (CSSTRPFY). Any key the screen does not accept is treated as ENTER. These screens never show the "invalid key" message.
- **BR-AC-02.** Commarea: the 160-byte COCOM01Y area, followed by a program-specific area. The whole commarea is passed back on RETURN (2000 bytes for COACTVWC).
- **BR-AC-03.** Entry from the main menu (`CDEMO-FROM-PROGRAM = COMEN01C` and the context is not REENTER) **re-initialises the whole commarea**. That clears the user id and user type, and the program then sets the user type to `U` before leaving with PF3.
- **BR-AC-04.** PF3 returns to `CDEMO-FROM-PROGRAM`/`CDEMO-FROM-TRANID` (or the main menu if they are blank). It sets from-tran and from-program to this program, sets context ENTER, sets the user type to `U`, and sets last mapset/map.
- **BR-AC-05.** File error messages use the format `File Error: <op 8> on <file 9> returned RESP <resp 10>,RESP2 <resp2 10>`. RESP values are printed as 9-digit unsigned numbers.
- **BR-AC-06.** An unexpected state sends plain text `UNEXPECTED DATA SCENARIO` and ends the task. An abend sends the abend data and abends with code 9999.
- **BR-AC-07.** A mandatory field left blank on re-entry is shown as `*` in red. A field in error is shown in red with the cursor on it.

4.1 FR-ACV — Account view (CAVW, COACTVWC, map COACTVW/CACTVWA)

ID	Requirement
FR-ACV-01	First entry: show the screen with the info message `Enter or update id of account to display`.
FR-ACV-02	Input: account number (11). Blank or `*` → `No input received` (the prompt `Account number not provided` is set first and then overwritten) and the field is shown as a red `*`. Non-numeric or all zeros → `Account Filter must  be a non-zero 11 digit number` (two spaces before `be`).
FR-ACV-03	Lookup chain: XREF by account (CXACAIX) gives the customer id and card number (also stored in the commarea). Then the account master, then the customer master. Not-found messages (RESP/RESP2 as 9 digits left-justified in 10-byte fields; the result is cut at 75 characters): `Account:<id> not found in Cross ref file.  Resp:<resp> Reas:<resp2>`, `Account:<id> not found in Acct Master file.Resp:<resp> Reas:<resp2>`, `CustId:<id> not found in customer master.Resp: <resp> REAS:<resp2>`.

ID	Requirement
FR-ACV-04	Displayed fields: account status, current balance, credit limit, cash credit limit, current cycle credit, current cycle debit, open, expiry and reissue dates, group id. Customer fields: id, SSN as `999-99-9999`, FICO score, date of birth, first, middle and last name, address lines 1–2, city (address line 3), state, zip, country, phone 1, phone 2, government id, EFT account id, primary card holder flag.
FR-ACV-06	*Map quirk:* map CACTVWA contains an unnamed field `LENGTH=9,POS=(1,1)` after the header fields. On a 3270 it overwrites row 1, columns 1–10 with nulls, so only the last letter of the transaction id (`W`) remains visible.
FR-ACV-05	PF3 → back to the caller (BR-AC-04).

4.2 FR-CDV — Credit card detail view (CCDL, COCRDSLC, map COCRDSL/CCRDSLA)

ID	Requirement
FR-CDV-01	Entered from the card list (context ENTER, from-program COCRDLIC): the account and card from the commarea are used to read the card immediately. Both search fields are then protected.
FR-CDV-02	Entered any other way: the prompt `Please enter Account and Card Number` is shown in the info line.
FR-CDV-03	Account filter: blank, `*` or all zeros → `Account number not provided`. Non-numeric → `ACCOUNT FILTER,IF SUPPLIED MUST BE A 11 DIGIT NUMBER`.
FR-CDV-04	Card filter: blank, `*` or all zeros → `Card number not provided`. Non-numeric → `CARD ID FILTER,IF SUPPLIED MUST BE A 16 DIGIT NUMBER`. Only the first error message is kept, except that when both filters are blank the message becomes `No input received`. Both fields are always validated, so both can be flagged red.
FR-CDV-05	If both filters are valid, the card is read by card number only; the account number is not cross-checked. Not found → `Did not find cards for this search condition` (both fields red). Found → info message `Displaying requested details` (three leading spaces), embossed name, expiry month and year (from `YYYY-MM-DD`), and card status (Y/N).
FR-CDV-06	PF3 → back to the caller (BR-AC-04). Other keys are treated as ENTER.

4.3 FR-CDL — Credit card list (CCLI, COCRDLIC, map COCRDLI/CCRDLIA)

ID	Requirement
FR-CDL-01	Shows the card file (CARDDAT, in card-number order) **7 rows per page**. Each row has a select field, account number, card number and active status. Optional filters: account id (11) and card number (16). A card is listed only if it matches every filter that was supplied.
FR-CDL-02	Filter validation: blank or zero means no filter. A non-numeric account → `ACCOUNT FILTER,IF SUPPLIED MUST BE A 11 DIGIT NUMBER`. A non-numeric card → `CARD ID FILTER,IF SUPPLIED MUST BE A 16 DIGIT NUMBER` (only if no earlier message). On any filter error the field turns red, the row select fields are protected, and the list is not re-read.
FR-CDL-03	Row action codes: `S` = view detail (XCTL COCRDSLC), `U` = update (XCTL COCRDUPC). The selected row's account and card number go into the commarea, with from-program COCRDLIC and context ENTER. More than one `S`/`U` → `PLEASE SELECT ONLY ONE RECORD TO VIEW OR UPDATE`, with the selected rows flagged red. Any other non-blank code → `INVALID ACTION CODE` (that row red).

ID	Requirement
FR-CDL-04	PF8: if there is a next page, show the 7 matching cards after the last card on the current page. With no further page → `NO MORE PAGES TO DISPLAY` once the last page has been shown. Reaching end of file while filling a page → `NO MORE RECORDS TO SHOW`.
FR-CDL-05	PF7: on the first page → `NO PREVIOUS PAGES TO DISPLAY`. Otherwise read backwards from the first card of the current page to fill the previous page.
FR-CDL-06	No matching card on the first page → `NO RECORDS FOUND FOR THIS SEARCH CONDITION.`
FR-CDL-07	When there are rows or more pages, the info line shows `TYPE S FOR DETAIL, U TO UPDATE ANY RECORD` (neutral).
FR-CDL-08	PF3 → main menu (from-tran `CCLI`, from-program `COCRDLIC`, user type `U`). Keys other than ENTER/PF3/PF7/PF8 are treated as ENTER. On entry from another program the list starts again from the first card.
FR-CDL-09	File errors use the BR-AC-05 format (here starting with the literal `File Error:` with no trailing space).

4.4 FR-CDU — Credit card update (CCUP, COCRDUPC, map COCRDUP/CCRDUPA)

This screen runs as a state machine. The state (`CCUP-CHANGE-ACTION`) is kept in the commarea extension, together with the old and new card details:

State	Meaning
not fetched (blank)	Waiting for the search keys.
`S`	Details shown.
`E`	Changes entered but not valid.
`N`	Changes valid, waiting for PF5.
`C`	Changes committed.
`L`	Lock error.
`F`	Update failed.

ID	Requirement
FR-CDU-01	Entered from the card list (or PF12 when called from the list): read the card for the account and card from the commarea and show its details (state S). Entered from the menu or with no details fetched: prompt `Please enter Account and Card Number`.
FR-CDU-02	Search key validation is the same as FR-CDV-03/04: `Account number not provided`, `ACCOUNT FILTER,IF SUPPLIED MUST BE A 11 DIGIT NUMBER`, `Card number not provided`, `CARD ID FILTER,IF SUPPLIED MUST BE A 16 DIGIT NUMBER`. Card not found → `Did not find cards for this search condition`. Found → info `Details of selected card shown above`. The embossed name is converted to upper case for display and comparison.
FR-CDU-03	Editable fields in state S/E: embossed name, active status, expiry month, expiry year. The expiry day is not editable (shown dark). The key fields are protected.
FR-CDU-04	If the new data equals the old data (case-insensitive) → `No change detected with respect to values fetched.`

ID	Requirement
FR-CDU-05	Validation (all four fields are checked; the first message is kept). Name: blank → `Card name not provided`; letters and spaces only, else `Card name can only contain alphabets and spaces`. Status: must be `Y` or `N`, else `Card Active Status must be Y or N`. Month: must be 1–12, else `Card expiry month must be between 1 and 12`. Year: must be 1950–2099, else `Invalid card expiry year`. Any failure → state E, info `Update card details presented above.`, and the failing fields shown in red (or `*` if blank).
FR-CDU-06	All valid → state N, info `Changes validated.Press F5 to save`. All fields are protected and the PF5 hint is highlighted.
FR-CDU-07	PF5 in state N: re-read the card FOR UPDATE. Failure → `Could not lock record for update` (state L). If the record differs from the old snapshot (CVV, name, expiry, status) → `Record changed by some one else. Please review`, and the new values are shown (state S). Otherwise REWRITE the card with: card number, account id, CVV (see KD-07), the new name, expiry `YYYY-MM-DD` from the new year and month plus the old day, and the new status. Rewrite failure → `Update of record failed` (state F). Success → state C, info `Changes committed to database`.
FR-CDU-08	After C/L/F (`Changes unsuccessful. Please try again` for L/F): if the screen was called from the card list, return to it automatically on the next key. Otherwise the next interaction starts again with an empty search.
FR-CDU-09	PF3 → back to the caller (BR-AC-04), with SYNCPOINT. If the caller was the card list, the commarea account and card are cleared. PF12 (once details are fetched) → re-fetch and discard the changes. Keys not valid in the current state are treated as ENTER.
KD-07	*Known defect, kept:* the CVV written on REWRITE comes from a field that is never filled (`CCUP-NEW-CVV-CD`), so an update **overwrites the card's CVV**: the field holds spaces (INITIALIZE), which `MOVE CARD-CVV-CD-N TO CARD-UPDATE-CVV-CD` copies, giving three spaces. This is what GnuCOBOL does and IBM COBOL with NUMPROC(PFD); with NUMPROC(NOPFD) IBM would also set the last digit's zone (`0`). The Rust port writes three spaces.

4.5 FR-ACU — Account update (CAUP, COACTUPC, map COACTUP/CACTUPA)

This screen uses the same state machine as FR-CDU (`ACUP-CHANGE-ACTION`: not fetched / S / E / N / C / L / F). One screen updates both the **account** record and its **customer** record.

ID	Requirement
FR-ACU-01	First entry: the info line shows `Enter or update id of account to update`. Only the account id field is enterable.
FR-ACU-02	Account id: blank or `*` → `No input received` (overwrites the prompt `Account number not provided`). Non-numeric or zero → `Account Number if supplied must be a 11 digit Non-Zero Number`.
FR-ACU-03	Fetch: read XREF by account (CXACAIX) to get the customer id, then the account master, then the customer master. The not-found messages are the same as FR-ACV-03. Success → state S, info `Details of selected account shown above`. The old values are kept in the commarea.
FR-ACU-04	Editable fields in state S/E. Account: active status, open date (Y/M/D), credit limit, expiry date (Y/M/D), cash credit limit, reissue date (Y/M/D), current balance, current cycle credit, current cycle debit, group id. Customer: SSN (3/2/4), date of birth (Y/M/D), FICO score, first, middle and last name, address line 1, address line 2, state, zip, city, country, phone 1 (area/prefix/line), phone 2, government id, EFT account id, primary card holder flag. A field containing `*` or spaces is treated as empty.

ID	Requirement
FR-ACU-05	Monetary inputs are accepted in any NUMVAL-C form (sign, commas, decimal point) and redisplayed with `+ZZZ,ZZZ,ZZZ.99`.
FR-ACU-06	If no field changed (case-insensitive for text) → `No change detected with respect to values fetched.` (never reached for a blank group id, KD-15).
FR-ACU-07	Validation runs in screen order and **every** field is checked. Only the **first** message is shown; every failing field turns red, or shows `*` if blank. `<Name>` below is the field label: Account Status, Open Date, Credit Limit, Expiry Date, Cash Credit Limit, Reissue Date, Current Balance, Current Cycle Credit Limit, Current Cycle Debit Limit, SSN, Date of Birth, FICO Score, First Name, Middle Name, Last Name, Address Line 1, State, Zip, City, Country, Phone Number 1, Phone Number 2, EFT Account Id, Primary Card Holder.
FR-ACU-07a	Yes/No fields (status, primary holder): blank → `<Name> must be supplied.`; not Y/N → `<Name> must be Y or N.`
FR-ACU-07b	Dates (CSUTLDPY rules). Year blank → `<Name> : Year must be supplied.`; non-numeric → `<Name> must be 4 digit number.`; century not 19/20 → `<Name> : Century is not valid.`. Month blank → `<Name> : Month must be supplied.`; not 1–12 → `<Name>: Month must be a number between 1 and 12.`. Day blank → `<Name> : Day must be supplied.`; not 1–31 → `<Name>:day must be a number between 1 and 31.`. Day 31 in a 30-day month → `<Name>:Cannot have 31 days in this month.`. Feb 30 → `<Name>:Cannot have 30 days in this month.`. Feb 29 in a non-leap year → `<Name>:Not a leap year.Cannot have 29 days in this month.`. Finally a CSUTLDTC check: `<Name> validation error Sev code: <sev> Message code: <msg>`. Date of birth must also be in the past: `Date of Birth:cannot be in the future`.
FR-ACU-07c	Signed amounts: blank → `<Name> must be supplied.`; not a valid NUMVAL-C → `<Name> is not valid`.
FR-ACU-07d	SSN part 1 (3 digits): required, numeric, non-zero (`SSN: First 3 chars must be supplied.`, `… must be all numeric.`, `… must not be zero.`), and not 000, 666 or 900–999 (`SSN: First 3 chars: should not be 000, 666, or between 900 and 999`). Part 2 (2 digits) and part 3 (4 digits): required, numeric, non-zero (`SSN 4th & 5th chars …`, `SSN Last 4 chars …`).
FR-ACU-07e	FICO: required numeric non-zero (3 digits), and between 300 and 850 → `FICO Score: should be between 300 and 850`.
FR-ACU-07f	Alphabetic fields: first name, last name, state, city and country are required (`<Name> must be supplied.`) and may contain only letters and spaces (`<Name> can have alphabets only.`). Middle name is optional but alphabetic. Address line 1 is mandatory (any characters). Address line 2 is not validated.
FR-ACU-07g	State must be a valid US state code (CSLKPCDY list) → `State: is not a valid state code`. Zip: required, numeric, non-zero (5 digits). If both state and zip are valid, the state plus the first 2 zip digits must be a valid combination (CSLKPCDY) → `Invalid zip code for state`.
FR-ACU-07h	Phones (area 3 / prefix 3 / line 4). An all-blank phone is accepted (see KD-08). Area code: required (`<Name>: Area code must be supplied.`), numeric (`…: Area code must be A 3 digit number.`), non-zero (`…: Area code cannot be zero`), and a valid North-American general-purpose area code from the CSLKPCDY list (`…: Not valid North America general purpose area code`). Prefix: required, numeric, non-zero (`…: Prefix code must be supplied.` / `must be A 3 digit number.` / `cannot be zero`). Line number: required, numeric, non-zero (`…: Line number code must be supplied.` / `must be A 4 digit number.` / `cannot be zero`).
FR-ACU-07i	EFT account id: required, numeric, non-zero (10 digits).
FR-ACU-08	Any validation error → state E, info `Update account details presented above.` All valid → state N, info `Changes validated.Press F5 to save`, all fields protected.

ID	Requirement
FR-ACU-09	PF5 in state N. Read the account FOR UPDATE (`Could not lock account record for update` → L), then the customer FOR UPDATE (`Could not lock customer record for update` → L). Compare both records with the old snapshot (case-insensitive for text fields); any difference → `Record changed by some one else. Please review` (state S). Otherwise build the updated records: dates as `YYYY-MM-DD`, phones as `(aaa)bbb-cccc`, SSN as 9 digits. REWRITE the account, then the customer. If the account rewrite fails → `Update of record failed` (F). If the customer rewrite fails → SYNCPOINT ROLLBACK (which also undoes the account) and F. Success → state C, info `Changes committed to database`.
FR-ACU-10	After C/L/F the next interaction starts with an empty search. The info line shows `Changes unsuccessful. Please try again` for L/F.
FR-ACU-11	PF3 → back to the caller with SYNCPOINT (BR-AC-04). PF12 (after a fetch) → re-read and discard edits. PF5 is only valid in state N; all other keys are treated as ENTER.
FR-ACU-12	*Data note:* the shipped sample customers have random FICO scores, area codes and zip codes. 48 of 50 fail the state/zip check and about half fail FICO or area-code checks. An update of these accounts is therefore only accepted after those fields are corrected.
KD-08	*Known defect, kept:* the “phone entirely blank” test checks the area code twice (once instead of the line number), so a phone with blank area and prefix but a non-blank line number is accepted as blank.
KD-13	*Known defect, kept:* COACTUPC rewrites the account from ACCT-UPDATE-RECORD, which lacks the ACCT-ADDR-ZIP field of CVACT01Y. The group id is therefore written at offset 102 (the master’s ADDR-ZIP) and ACCT-GROUP-ID plus the FILLER become X’00’. A blank group input is stored as LOW-VALUES. After an update the account view shows an empty group, and INTCALC finds no matching discount group. (The shipped sample data already holds its group values in the ADDR-ZIP position.)
KD-15	*Known defect, kept:* the no-change test compares the group id with `UPPER-CASE(TRIM(...))`. A blank group input is stored as LOW-VALUES, which TRIM does not remove, while the old value is spaces, so an account with a blank group id (all sample accounts) never compares as unchanged: Enter without changes validates all fields instead of reporting `No change detected...`.
KD-16	*Undefined behaviour in the original:* COACTVWC, COACTUPC, COCRDLIC, COCRDSLC and COCRDUPC copy their program area from `DFHCOMMAREA` beyond `EIBCALEN` even when the caller passed only CARDDEMO-COMMAREA (e.g. card list → card update). On CICS this reads whatever storage follows the commarea; in card update those bytes reach the FILLER of the card record written on PF5. The Rust port and the COBOL bridge both supply binary zeros there.

5 Online Requirements — Transactions, Bill Payment, Reports

Transaction master: TRANSACT KSDS keyed by TRAN-ID (16 characters), record CVTRA05Y (350 bytes).

5.1 FR-TRL — Transaction list (CT00, COTRN00C, map COTRN00/COTRN0A)

ID	Requirement
FR-TRL-01	Shows up to 10 transactions per page in TRAN-ID order. Each row has a select field, transaction id, date (`mm/dd/yy` from TRAN-ORIG-TS), description and amount (`+99999999.99`). The page number is shown.
FR-TRL-02	Optional “Search Tran ID” field. If it is blank, browsing starts at the first record. If it is non-numeric → `Tran ID must be Numeric ...`. Otherwise browsing starts at that key (STARTBR GTEQ).

ID	Requirement
FR-TRL-03	Selection `S`/`s` on a row → XCTL COTRN01C with the selected id in the commarea extension (CDEMO-CT00-TRN-SELECTED). Any other code → `Invalid selection. Valid value is S`.
FR-TRL-04	Paging with PF7/PF8 works exactly like FR-USR-LST-04..06, with these messages: `You are already at the top of the page...`, `You are already at the bottom of the page...`, `You have reached the bottom of the page...`, `You have reached the top of the page...`, `You are at the top of the page...` (STARTBR not found).
FR-TRL-05	PF3 → main menu (COMEN01C). File errors → `Unable to lookup transaction....`

5.2 FR-TRV — Transaction view (CT01, COTRN01C, map COTRN01/COTRN1A)

ID	Requirement
FR-TRV-01	On first entry with a transaction preselected (from the list), it is displayed immediately.
FR-TRV-02	ENTER: blank id → `Tran ID can NOT be empty....` Not found → `Transaction ID NOT found....` Other error → `Unable to lookup Transaction....`
FR-TRV-03	Displays transaction id, card number, type code, category code, source, amount (`+99999999.99`), description, original and processed timestamps, merchant id, merchant name, city and zip.
FR-TRV-04	PF3 → back to the caller (or the main menu). PF4 → clear the screen. PF5 → transaction list (COTRN00C).

5.3 FR-TRA — Transaction add (CT02, COTRN02C, map COTRN02/COTRN2A)

Processing rule BR-TRA-0: in this program every screen send also ends the task (SEND followed by RETURN TRANSID). So the **first** validation error found is the one shown, and nothing after it runs.

ID	Requirement
FR-TRA-01	Key fields: account id **or** card number. If an account id is entered: it must be numeric (`Account ID must be Numeric...`); it is looked up in the XREF alternate index by account (`Account ID NOT found...`); and the card number field is filled from XREF. Otherwise, if a card number is entered: it must be numeric (`Card Number must be Numeric...`); it is looked up in XREF (`Card Number NOT found...`); and the account id is filled. If neither is entered → `Account or Card Number must be entered....`
FR-TRA-02	Mandatory data fields, checked in this order: Type CD, Category CD, Source, Description, Amount, Orig Date, Proc Date, Merchant ID, Merchant Name, Merchant City, Merchant Zip. Each gives `<Field> can NOT be empty...` (e.g. `Type CD can NOT be empty...`, `Merchant Zip can NOT be empty...`).
FR-TRA-03	Type CD and Category CD must be numeric (`Type CD must be Numeric...`, `Category CD must be Numeric...`).
FR-TRA-04	Amount format: sign `+`/`-`, 8 digits, `.`, 2 digits. Otherwise → `Amount should be in format -99999999.99`. The amount is then normalised and redisplayed as `+99999999.99`.
FR-TRA-05	Orig and Proc dates must have the form `YYYY-MM-DD` (`Orig Date should be in format YYYY-MM-DD`, `Proc Date ...`). They must also be valid calendar dates according to CSUTLDTC (`Orig Date - Not a valid date...`, `Proc Date - Not a valid date...`). Message number 2513 is tolerated.
FR-TRA-06	Merchant ID must be numeric (`Merchant ID must be Numeric...`).

ID	Requirement
FR-TRA-07	Confirmation field: Y/y → add. N/n/blank → `Confirm to add this transaction...`. Anything else → `Invalid value. Valid values are (Y/N)...`.
FR-TRA-08	Add: find the highest existing TRAN-ID (browse from HIGH-VALUES, READPREV; an empty file gives 0). The new id is that value + 1, as 16 digits. Build the record from the screen: type, category, source, description, amount, card number, merchant fields, orig TS = the orig date, proc TS = the proc date. Then WRITE it.
FR-TRA-09	Success → all fields cleared, message `Transaction added successfully. Your Tran ID is <id>.` in green. Duplicate key → `Tran ID already exist...`. Other error → `Unable to Add Transaction...`.
FR-TRA-10	PF5 copies the data fields of the **last transaction in the file** (highest id) into the screen, after validating the key fields, and then runs the ENTER processing. PF4 clears. PF3 → back to the caller (or the main menu).

5.4 FR-BIL — Bill payment (CB00, COBIL00C, map COBIL00/COBIL0A)

ID	Requirement
FR-BIL-01	Input: account id (11) and a confirmation (Y/N). Blank account id → `Acct ID can NOT be empty...`. The account id is not checked for digits before it is moved to the PIC 9(11) key, so any other input is compiler-dependent (GnuCOBOL reads 6 followed by spaces as account 6; the Rust port uses the characters as typed, so that account is not found).
FR-BIL-02	The account is read for update. Not found → `Account ID NOT found...`. Other error → `Unable to lookup Account...`. The current balance is shown as `+9999999999.99`.
FR-BIL-03	If the balance is ≤ 0 → `You have nothing to pay...`.
FR-BIL-04	Confirmation blank → the balance is shown with `Confirm to make a bill payment...`. N/n → screen cleared. Any value other than Y/N → `Invalid value. Valid values are (Y/N)...`.
FR-BIL-05	Confirmation Y/y: read the XREF by account (AIX) to get the card number. The new transaction id is the highest TRAN-ID + 1. Write a transaction with type `02`, category `0002`, source `POS TERM`, description `BILL PAYMENT - ONLINE`, amount = **full current balance**, card number from XREF, merchant id `999999999`, merchant name `BILL PAYMENT`, city `N/A`, zip `N/A`, orig and proc timestamps = current `YYYY-MM-DD HH:MM:SS.000000`.
FR-BIL-06	After the write: `ACCT-CURR-BAL = ACCT-CURR-BAL − amount` (i.e. 0), then REWRITE the account. Success message (green): `Payment successful. Your Transaction ID is <id>.`, and the fields are cleared.
FR-BIL-07	Errors: `Tran ID already exist...`, `Unable to Add Bill pay Transaction...`, `Unable to Update Account...`, `Unable to lookup XREF AIX file...`, `Unable to lookup Transaction...`.
FR-BIL-08	PF3 → back to the caller (or the main menu). PF4 → clear.

5.5 FR-RPT — Transaction reports (CR00, CORPT00C, map CORPT00/CORPT0A)

Processing rule BR-RPT-0: every screen send ends the task (SEND then RETURN), so the first error found is the one shown.

ID	Requirement
FR-RPT-01	Choose one report type, checked in this order: Monthly, Yearly, Custom. If none is chosen → `Select a report type to print report...`.

ID	Requirement
FR-RPT-02	Monthly: from the 1st of the current month to its last day (computed as the 1st of the next month minus 1 day). Yearly: from `YYYY-01-01` to `YYYY-12-31` of the current year.
FR-RPT-03	Custom: start and end month, day and year are mandatory (`Start Date - Month can NOT be empty...`, `Start Date - Day ...`, `Start Date - Year ...`, `End Date - Month ...`, `End Date - Day ...`, `End Date - Year ...`). The values are normalised with NUMVAL-C into zero-padded numbers. Month > 12 or non-numeric → `Start Date - Not a valid Month...` (or End). Day > 31 → `... Not a valid Day....` Year non-numeric → `... Not a valid Year....` A calendar date check via CSUTLDTC gives `Start Date - Not a valid date...` / `End Date - Not a valid date....`
FR-RPT-04	Confirmation: blank → `Please confirm to print the <name> report....` N/n → screen cleared. Any other value except Y → `"<value>" is not a valid value to confirm....`
FR-RPT-05	On confirmation `Y`: the JCL deck for TRANREPT (with SORT symbol parameters for the start and end dates and the DATEPARM record `<start> <end>`) is written line by line to the TD queue `JOBS` (the internal reader). *Rust: the job is queued to the built-in job runner, which runs the TRANREPT job (FR-BAT-REPT) with those dates. The JCL text is kept in a spool log for traceability.* TDQ failure → `Unable to Write TDQ (JOBS)....`
FR-RPT-06	Success → fields cleared, message (green) `<Monthly|Yearly|Custom> report submitted for printing ....`
FR-RPT-07	PF3 → main menu.

6 Batch Processing Requirements

All batch programs are sequential jobs. Each one opens its files at start. If a file operation returns an unexpected status, the program displays a diagnostic and abends with user code 999 (`CEE3ABD`). That is a non-zero termination, and later steps in the job are skipped. The status diagnostic has the form `FILE STATUS IS: NNNNxxxx`, where `xxxx` is the 4-digit status (`00ss` for numeric statuses; for VSAM `9x` statuses it is 9 followed by the binary value of the second byte as 3 digits).

Common rule **BR-BAT-001**: each program displays `START OF EXECUTION OF PROGRAM <name>` and `END OF EXECUTION OF PROGRAM <name>`. (CBEXPORT and CBIMPORT use their own banners instead.)

Common rule **BR-BAT-002**: when a program builds a "DB2-format timestamp", the format is `YYYY-MM-DD-HH.MM.SS.cc0000`. The date parts are separated by `-`, the time parts by `.`, `cc` is hundredths of a second, and the value ends in the literal `0000`. Total length 26.

6.1 FR-BAT-POST — Daily transaction posting (job POSTTRAN, program CBTRN02C)

Source: cobol/app/cbl/CBTRN02C.cbl, cobol/app/jcl/POSTTRAN.jcl

Inputs: DALYTRAN (sequential, 350-byte CVTRA06Y records), XREFFILE (KSDS keyed by card number), ACCTFILE (KSDS keyed by account id, opened I-O), TCATBALF (KSDS keyed by account id + type code + category code, opened I-O). **Outputs:** TRANSACT (KSDS keyed by transaction id, opened OUTPUT, so the master file is reloaded with only the posted records), DALYREJS (sequential, 430 bytes: the 350-byte input record plus an 80-byte validation trailer).

ID	Requirement
FR-BAT-POST-01	Read every DALYTRAN record in sequence and count it in *transactions processed*.
FR-BAT-POST-02	Validation step 1: look up the card number in XREF. If it is not found, set reason **100** `INVALID CARD NUMBER FOUND` and skip the remaining checks.

ID	Requirement
FR-BAT-POST-03	Validation step 2: read the account by the XREF account id. If it is not found, set reason **101** `ACCOUNT RECORD NOT FOUND`.
FR-BAT-POST-04	Over-limit check: compute `temp = ACCT-CURR-CYC-CREDIT − ACCT-CURR-CYC-DEBIT + DALYTRAN-AMT`. If `ACCT-CREDIT-LIMIT < temp`, set reason **102** `OVERLIMIT TRANSACTION`.
FR-BAT-POST-05	Expiry check: if `ACCT-EXPIRAION-DATE < DALYTRAN-ORIG-TS(1:10)` (string comparison), set reason **103** `TRANSACTION RECEIVED AFTER ACCT EXPIRATION`. Checks 04 and 05 are both evaluated, and the later one overwrites the reason code.
FR-BAT-POST-06	Rejected records go to DALYREJS: the input record followed by the trailer (reason code `9(04)` + description `X(76)`), and are counted in *transactions rejected*.
FR-BAT-POST-07	Posting builds a TRAN record from the daily record field by field. `TRAN-PROC-TS` is the current DB2-format timestamp (BR-BAT-002).
FR-BAT-POST-08	Category balance: read TCATBAL by (xref account id, type, category). If it is not found, display `TCATBAL record not found for key : <key>.. Creating.` and write a new record with balance = amount. Otherwise add the amount and rewrite.
FR-BAT-POST-09	Account update: `ACCT-CURR-BAL += amt`. If `amt ⩾ 0`, `ACCT-CURR-CYC-CREDIT += amt`; otherwise `ACCT-CURR-CYC-DEBIT += amt`. Then rewrite the account.
FR-BAT-POST-10	Write the TRAN record to TRANSACT. A duplicate key or other error abends.
FR-BAT-POST-11	At end, display `TRANSACTIONS PROCESSED :nnnnnnnnn` and `TRANSACTIONS REJECTED :nnnnnnnnn` (counters are `9(09)`). If any record was rejected, set return code **4**.

6.2 FR-BAT-INT — Interest calculation (job INTCALC, program CBACT04C)

Source: cobol/app/cbl/CBACT04C.cbl, cobol/app/jcl/INTCALC.jcl. PARM = run date `YYYY-MM-DD` (e.g. `2022071800`, of which the first 10 characters are used).

Inputs: TCATBALF (KSDS, read sequentially in key order), XREFFILE (KSDS, read through the alternate key, account id), DISCGRP (KSDS keyed by group id + type + category), ACCTFILE (I-O). **Output:** TRANSACT (sequential, a new SYSTRAN GDG generation) with one interest transaction per category balance that has a non-zero rate.

ID	Requirement
FR-BAT-INT-01	Process TCATBAL records in key order. Display each record.
FR-BAT-INT-02	When the account id changes, rewrite the previous account (FR-BAT-INT-06). On the first record, only set the first-time flag. Then reset total interest to 0, read the account (`ACCOUNT NOT FOUND: <id>` if missing) and read the XREF by account id.
FR-BAT-INT-03	Interest rate: read DISCGRP with (ACCT-GROUP-ID, type, category). If it is not found (status 23), display `DISCLOSURE GROUP RECORD MISSING` / `TRY WITH DEFAULT GROUP CODE` and re-read with group id `DEFAULT`. If that also fails, abend.
FR-BAT-INT-04	If the rate is not 0: `monthly = (TRAN-CAT-BAL × DIS-INT-RATE) / 1200`, truncated to 2 decimals (`S9(09)V99`, no ROUNDED). Add it to total interest.
FR-BAT-INT-05	For each computed interest, write a transaction: ID = PARM date (10 chars) + 6-digit running suffix starting at 000001; type `01`; category `0005`; source `System`; description `Int. for a/c` + ACCT-ID (11 digits); amount = monthly interest; merchant id 0; merchant name/city/zip blank; card = XREF card number; orig and proc timestamps = current DB2-format timestamp.
FR-BAT-INT-06	Account update: `ACCT-CURR-BAL += total interest`, then `ACCT-CURR-CYC-CREDIT = 0` and `ACCT-CURR-CYC-DEBIT = 0`, then rewrite.
FR-BAT-INT-07	Fees (`1400-COMPUTE-FEES`) are a stub that does nothing.

ID	Requirement
KD-01	*Known defect, kept for fidelity:* the end-of-file `ELSE PERFORM 1050-UPDATE-ACCOUNT` branch can never be reached, so the **last account** in TCATBAL never gets its interest and cycle reset written back. Its interest transactions are still written.

6.3 FR-BAT-REPT — Transaction detail report (job TRANREPT, program CBTRN03C)

Source: cobol/app/cbl/CBTRN03C.cbl, cobol/app/jcl/TRANREPT.jcl. The JCL first sorts TRANSACT by card number (and filters by date) into TRANSACT.DALY, then runs CBTRN03C. The job can also be submitted from the online Report screen (CORPT00C).

Inputs: TRANFILE (sequential, sorted), CARDXREF, TRANTYPE, TRANCATG (KSDS), DATEPARM (an 80-byte record: start date `X(10)`, filler, end date `X(10)`). **Output:** TRANREPT (133-byte lines).

ID	Requirement
FR-BAT-REPT-01	Read DATEPARM once and display `Reporting from <start> to <end>`.
FR-BAT-REPT-02	Read each transaction. If `TRAN-PROC-TS(1:10)` is outside [start, end], the record is skipped with NEXT SENTENCE. *Fidelity note: the exact GnuCOBOL/Enterprise COBOL semantics are verified by differential test (DT-REPT).*
FR-BAT-REPT-03	When the card number changes (and it is not the first record), write the account totals line. Then look up XREF for the new card. A missing XREF, transaction type, or category abends after `INVALID CARD NUMBER :`, `INVALID TRANSACTION TYPE :`, or `INVALID TRAN CATG KEY :`.
FR-BAT-REPT-04	On the first detail line, write the headers: name header (`DALYREPT` padded to 38, `Daily Transaction Report` padded to 41, `Date Range: <start> to <end>`), a blank line, column header 1, and a line of 133 `-`.
FR-BAT-REPT-05	Page size is 20 lines. When `line-counter mod 20 = 0` before a detail line, write the page totals and repeat the headers.
FR-BAT-REPT-06	Detail line layout (CVTRA07Y): tran id(16) sp acct(11) sp type(2) - type-desc(15) sp cat(4) - cat-desc(29) sp source(10) 4sp amount `-ZZZ,ZZZ,ZZZ.ZZ` 2sp.
FR-BAT-REPT-07	Totals: page total `Page Total` + dots + `+ZZZ,ZZZ,ZZZ.ZZ`. The page total adds to the grand total and resets. The account total resets after printing. Each totals line is followed by a dashed line.
FR-BAT-REPT-08	At EOF: add the last record's amount to the page and account totals **again** (KD-02), then write page totals and grand totals. The last account's totals line is not written (KD-03).

6.4 FR-BAT-STMT — Customer statements (job CREASTMT, programs CBSTM03A + CBSTM03B)

Source: cobol/app/cbl/CBSTM03A.CBL, CBSTM03B.CBL, cobol/app/jcl/CREASTMT.JCL.

The job sorts TRANSACT by card number (pos 263, length 16) then transaction id, and loads the result into a KSDS (TRXFL) keyed by card + tran id. CBSTM03A produces two outputs: STMTFILE, a plain-text statement in 80-byte lines, and HTMLFILE, an HTML statement in 100-byte lines. CBSTM03B is a file-access subroutine. CBSTM03A calls it with a DD name, an operation (`O` open, `C` close, `R` read next, `K` read by key), a key and key length, and receives a 2-character return code and the record data.

ID	Requirement
FR-BAT-STMT-01	(z/OS-only diagnostics) display the job and step name and the list of DD names from the TIOT. *Rust: the job/step name and the configured DD mapping.*

ID	Requirement
FR-BAT-STMT-02	Load **all** transactions into memory, grouped by card: at most 51 cards × 10 transactions per card. Overflowing these limits is undefined in the original (KD-04).
FR-BAT-STMT-03	For each XREF record in card order: read the customer by XREF cust id and the account by XREF acct id (either missing → abend), then produce one statement.
FR-BAT-STMT-04	Text statement lines: `*` ×31 + `START OF STATEMENT` + `*` ×31; name (first, middle, last, each trimmed at the first space and joined with single spaces); address line 1; address line 2; line 3 + state + country + zip (each trimmed and space-joined); dashes; `Basic Details` centred; dashes; `Account ID :` padded to 20 + acct id; `Current Balance :` + `9(9).99-`; `FICO Score :` + score; dashes; `TRANSACTION SUMMARY`; dashes; column headings; dashes.
FR-BAT-STMT-05	Each transaction of the card: tran id(16) sp description(49) $ amount `Z(9).99-`. Its amount adds to the total.
FR-BAT-STMT-06	Trailer: dashes, `Total EXP:` + 56 spaces + $ + total `Z(9).99-`, then `*` ×32 + `END OF STATEMENT` + `*` ×32.
FR-BAT-STMT-07	The HTML statement has the same content as an HTML table. Fixed lines come from the 88-level literals (bank name `Bank of XYZ`, address `410 Terry Ave N`, `Seattle WA 99999`). Each statement is a complete `<!DOCTYPE html>` … `</html>` document appended to the same file.

6.5 FR-BAT-RDR — File dump utilities (READACCT/CBACT01C, READCARD/CBACT02C, READXREF/CBACT03C, READCUST/CBCUS01C)

ID	Requirement
FR-BAT-RDR-01	CBACT02C, CBACT03C and CBCUS01C read the card, XREF and customer KSDS sequentially and DISPLAY each record. CBACT03C and CBCUS01C display each record twice (once in the read paragraph, once in the main loop).
FR-BAT-RDR-02	CBACT01C reads accounts sequentially and, for each one, displays the labelled fields (`ACCT-ID :` … `ACCT-GROUP-ID :`, then a dashed line), then the VBRC records, and finally the whole account record (main-loop DISPLAY). It writes three outputs: (a) OUTFILE: id, status, balance, limits, dates; reissue date converted from `YYYY-MM-DD` to `YYYYMMDD` by COBDATFT (the 10-byte field ends in two X'00' bytes: COBDATFT stores 8 bytes into a WORKING-STORAGE area without VALUE); cycle credit; cycle debit as COMP-3, set to `2525.00` when the account's debit is zero and otherwise left unchanged from the previous record (KD-05); group id. (b) ARRYFILE: id + 5 × (balance, COMP-3 debit). Slot 1 = (bal, 1005.00), slot 2 = (bal, 1525.00), slot 3 = (—1025.00, —2500.00), slots 4–5 = zero. (c) VBRCFILE (variable-length records): a 12-byte record (id + status) and a 39-byte record (id + balance + limit + reissue year).
FR-BAT-RDR-03	COBDATFT (assembler): type 1 input `YYYYMMDD` → output `YYYY-MM-DD` (output type must be 1); type 2 input `YYYY-MM-DD` → output `YYYYMMDD` (output type must be 2). Anything else sets the error message `INVALID INPUT`.

6.6 FR-BAT-VER — Daily transaction verification (CBTRN01C; no JCL ships with the original)

ID	Requirement
FR-BAT-VER-01	Read DALYTRAN sequentially and DISPLAY each record. For each record: look up XREF by card number. If found, display `SUCCESSFUL READ OF XREF`, `CARD NUMBER:`, `ACCOUNT ID :`, `CUSTOMER ID:`, then read the account (`SUCCESSFUL READ OF ACCOUNT FILE`, or `INVALID ACCOUNT NUMBER FOUND` + `ACCOUNT <id> NOT FOUND`). If not found, display `INVALID CARD NUMBER FOR XREF` and `CARD NUMBER <n> COULD NOT BE VERIFIED. SKIPPING TRANSACTION ID—<id>`. No files are updated.
FR-BAT-VER-02	After end of file the lookup runs once more for the last record read (loop structure of the original).

6.7 FR-BAT-XPRT — Export / import (CBEXPORT, CBIMPORT)

ID	Requirement
FR-BAT-XPRT-01	CBEXPORT reads customers, accounts, XREF, transactions and cards in that order. For each record it writes one 500-byte EXPORT-RECORD (CVEXPORT) with type `C`/`A`/`X`/`T`/`D`, timestamp `YYYY-MM-DD HH:MM:SS.00` followed by 4 bytes X'00' (STRING into a 26-byte field without VALUE), binary sequence number, branch `0001`, region `NORTH`, and the record body in the mixed COMP / COMP-3 / zoned layout of CVEXPORT.
FR-BAT-XPRT-02	It displays per-type counts and the total at the end.
FR-BAT-XPRT-03	CBIMPORT reads the export file and splits it by record type back into customer, account, XREF, transaction and card files. An unknown type is written to ERROUT as `timestamp|type|seq|Unknown record type encountered`.
FR-BAT-XPRT-05	*Deviation:* the EXPORT.DATA KSDS (`KEYS(4 28)` over a binary sequence number) is a fixed-length sequential file in write order. CBIMPORT reads it in the same order.
FR-BAT-XPRT-04	It displays the import statistics (read, per-type counts, errors written, unknown types). Output records are INITIALIZEd field by field, so their FILLER bytes are X'00': an export/import round trip reproduces every named field, not the FILLER bytes. Validation (`3000-VALIDATE-IMPORT`) only displays a completion message.

6.8 FR-BAT-UTIL — Utilities

ID	Requirement
FR-BAT-UTIL-01	COBSWAIT reads an 8-digit value from SYSIN and waits that many **centiseconds** (MVSWAIT STIMER BINTVL units).
FR-BAT-UTIL-02	CSUTLDTC validates a date string against a format mask (as CEEDAYS does) and returns an 80-byte message: severity `9(4)`, `Mesg Code:`, message number, result text (`Date is valid`, `Insufficient`, `Datevalue error`, `Invalid Era`, `Unsupp. Range`, `Invalid month`, `Bad Pic String`, `Nonnumeric data`, `YearInEra is 0`, `Date is invalid`), `TstDate:` + date, `Mask used:` + mask. RETURN-CODE = severity. Supported masks used by the application: `YYYY-MM-DD` and `YYYYMMDD`.
FR-BAT-UTIL-03	IDCAMS REPRO / DEFINE / DELETE, IEBGENER, SORT and IEFBR14 steps are replaced by the Rust job runner's built-in steps: load file into store, unload store to file, sort by key fields, delete/define dataset, GDG new generation.

6.9 Known defects kept for fidelity

ID	Program	Description
KD-01	CBACT04C	The last account's interest update is never written back.
KD-02	CBTRN03C	At EOF the last transaction amount is added to the totals a second time.
KD-03	CBTRN03C	The last account's "Account Total" line is never printed.
KD-04	CBSTM03A	Fixed in-memory limits of 51 cards × 10 transactions, with no bounds check.
KD-09	CBIMPORT JCL	The JCL has no CARDOUT DD although the program opens CARD-OUTPUT. The Rust job supplies it.
KD-05	CBACT01C	The OUTFILE cycle-debit value carries over from the previous record when the account's debit is non-zero.
KD-12	CBEXPORT, CBIMPORT	The export file's `RECORD KEY IS EXPORT-SEQUENCE-NUM` names a WORKING-STORAGE field (CVEXPORT), not a field of the file's own record. COBOL requires the key inside the record description, so neither program compiles as shipped (verified with GnuCOBOL 3.2). The Rust port implements the evident intent; the differential test compiles both with the key moved into the FD record at the same offset.

Source note: `CUSTREC.cpy` and `CSLKPCDY.cpy` contain TAB characters. With 8-column tabs the CUSTREC fields run past column 72 and CBSTM03A does not compile; the alignment of the other lines shows that 4-column tabs were intended.

7 Non-Functional Requirements

ID	Requirement
NFR-01	**Behavioural fidelity.** For the same input data and key strokes, the Rust system shows the same screens (field contents, messages, colours and cursor position) and writes byte-identical output records and reports. The only exceptions are run-dependent values: current date/time, timestamps, and system/application ids.
NFR-02	**Data compatibility.** The Rust system imports and exports the original fixed-width record layouts in both ASCII and EBCDIC (code page 037). That includes zoned decimal with overpunched sign, COMP-3 and binary fields. A load followed by an unload is byte-identical to the input.
NFR-03	**Decimal arithmetic.** All monetary arithmetic uses fixed-point decimal with the COBOL picture's precision. Results are truncated (not rounded) unless the source says ROUNDED. Binary floating point is never used for money.
NFR-04	**Transactional integrity.** Each online interaction (one pseudo-conversational task) is a unit of work. Updates are committed at task end or SYNCPOINT, and rolled back on SYNCPOINT ROLLBACK or abend. A batch step that abends leaves the files in the state the original would. If the unit of work cannot be opened (another terminal or job holds the store beyond the busy timeout), the task is not run and the screen says no updates were made; a batch step ends with abend U0998. A failed file operation leaves no partial alternate-index changes.
NFR-05	**Terminal UI.** The online application runs in a terminal at least 80×24, rendering each BMS map at its original row/column positions and colours. PF1–PF12 are F1–F12. PF13–PF24 are Shift+F1–F12. ENTER, CLEAR (Esc), PA1 and PA2 are supported. Field-level protection and numeric-only attributes are enforced.
NFR-06	**Multi-user.** Several terminal sessions can run against the same data store at the same time. Record locking makes READ UPDATE / REWRITE safe.

ID	Requirement
NFR-07	**Batch operation.** Each JCL job is available as a named job run from the command line, with the same step order, condition-code handling and return codes (0, 4, abend 999 → non-zero exit).
NFR-08	**Demonstrability.** One command resets the data store to the shipped sample data. A scripted demo (sign-on, browse, update, posting run, report) completes in under 10 minutes. The application starts in under 1 second.
NFR-09	**Portability.** The system builds with stable Rust on Linux and Windows. It needs no mainframe, CICS, VSAM or COBOL runtime. SQLite is embedded. Running the original COBOL programs for comparison (`--cobol`, the differential tests) needs GnuCOBOL and Linux.
NFR-10	**Traceability.** Each Rust program module names the COBOL program it replaces. Its functions are named after the original paragraphs.
NFR-11	**Security parity.** Passwords are stored and compared in plain text and upper case, as in the original (USRSEC). This is a known weakness kept for fidelity, and the deployment must treat the data store as sensitive.
NFR-12	**Observability.** Batch programs write their DISPLAY output to stdout and the job log, as SYSOUT did. Online abends are logged with the program name and abend code.
NFR-13	**Undefined storage.** Data items without a VALUE clause (WORKING-STORAGE and FD record areas) start as binary zeros, and INITIALIZE leaves FILLER and REDEFINES items unchanged, as Enterprise COBOL does. Where the original writes such bytes (FILLERs of posted and interest transactions, new TCATBAL records, import outputs, the export timestamp), the Rust port writes the same X'00' bytes.
NFR-14	**Verified against the original.** The batch programs are compiled from the original sources with GnuCOBOL and run on the same input as the Rust port (`rust/difftest/run.py`). Every output file, report, return code and SYSOUT line must be identical, apart from documented platform differences.
NFR-15	**Online verified against the original.** The original COBOL online programs run unchanged on a CICS bridge (GnuCOBOL + the project's CICS layer, `rust/cobridge`). The online tests run on both with the same keystrokes and data, and every screen (map, cursor, field contents, colours, attributes) must be identical (`rust/difftest/online.py`). Documented exceptions: invalid data moved into numeric fields, whose result depends on the compiler.

8 Traceability and Glossary

8.1 Program → requirement traceability

Source program / artefact	Requirement groups	Count
COSGN00C	FR-SGN-01 … FR-SGN-07	7
COMEN01C / COMEN02Y	FR-MEN-01 … FR-MEN-07	7
COADM01C / COADM02Y	FR-ADM-01 … FR-ADM-04	4
COUSR00C	FR-USR-LST-01 … FR-USR-LST-07	7
COUSR01C	FR-USR-ADD-01 … FR-USR-ADD-04	4
COUSR02C	FR-USR-UPD-01 … FR-USR-UPD-04	4
COUSR03C	FR-USR-DEL-01 … FR-USR-DEL-03	3
COACTVWC	FR-ACV-01 … FR-ACV-05	5
COCRDSLC	FR-CDV-01 … FR-CDV-06	6
COCRDLIC	FR-CDL-01 … FR-CDL-09	9
COCRDUPC	FR-CDU-01 … FR-CDU-09	9

Source program / artefact	Requirement groups	Count
COACTUPC (+CSUTLDPY, CSLKPCDY)	FR-ACU-01 … FR-ACU-11	20
COTRN00C	FR-TRL-01 … FR-TRL-05	5
COTRN01C	FR-TRV-01 … FR-TRV-04	4
COTRN02C (+CSUTLDTC)	FR-TRA-01 … FR-TRA-10	10
COBIL00C	FR-BIL-01 … FR-BIL-08	8
CORPT00C	FR-RPT-01 … FR-RPT-07	7
CBTRN02C / POSTTRAN	FR-BAT-POST-01 … FR-BAT-POST-11	11
CBACT04C / INTCALC	FR-BAT-INT-01 … FR-BAT-INT-07	7
CBTRN03C / TRANREPT	FR-BAT-REPT-01 … FR-BAT-REPT-08	8
CBSTM03A, CBSTM03B / CREASTMT	FR-BAT-STMT-01 … FR-BAT-STMT-07	7
CBACT01C-03C, CBCUS01C, COBDATFT / READxxxx	FR-BAT-RDR-01 … FR-BAT-RDR-03	3
CBTRN01C (DALYVRFY)	FR-BAT-VER-01 … FR-BAT-VER-02	2
CBEXPORT, CBIMPORT	FR-BAT-XPRT-01 … FR-BAT-XPRT-04	4
COBSWAIT+MVSWAIT, CSUTLDTC, IDCAMS/SORT/IEBGENER steps	FR-BAT-UTIL-01 … FR-BAT-UTIL-03	3

Totals: 162 functional requirements, 8 shared rules in tables, 12 non-functional requirements and 7 known-defect entries.

8.2 Glossary

Term	Meaning
AIX	VSAM alternate index (a secondary key path), e.g. CXACAIX = XREF by account id
BMS	CICS Basic Mapping Support. A *map* is a 24×80 screen layout; a *mapset* groups maps
COMMAREA	Communication area passed between CICS programs and between pseudo-conversational ta
COMP / COMP-3	Binary / packed-decimal numeric storage
KSDS	VSAM key-sequenced data set (indexed file)
Pseudo-conversational	Each screen interaction is a separate short task; state is carried in the COMMAREA
RESP / RESP2	CICS response codes (e.g. 13 = NOTFND, 14 = DUPREC, 20 = ENDFILE)
XCTL	CICS transfer of control to another program, with no return
Zoned decimal	One digit per byte; the sign is overpunched in the last byte (e.g. { = +0, } = —0)